APPENDIX B: 80x86 INSTRUCTIONS AND TIMING

In the first section of this appendix, we list the instructions of the 8086, give their format and expected operands, and describe the function of each instruction. Where pertinent, programming examples have been given. These instructions will operate on any 8086 or higher IBM-compatible computer. There are additional instructions for higher microprocessors (80186 and above); however, these instructions are not given in this list. The second section is a list of clock counts for each instruction across the 80x86 family.

SECTION B.1: THE 8086 INSTRUCTION SET

AAA ASCII Adjust after Addition

Flags: Affected: AF and CF. Unpredictable: OF, SF, ZF, PF.
Format: AAA
Function: This instruction is used after an ADD instruction has added two digits in ASCII code. This makes it possible to add ASCII numbers without masking off the upper nibble "3". The result will be unpacked BCD in AL with carry flag set if needed. This instruction adjusts only on the AL register. AH is incremented if the carry flag is set.

Example 1:

```
MOV   AL,31H      ;AL=31 THE ASCII CODE FOR 1
ADD   AL,37H      ;ADD 37 (ASCII FOR 7) TO AL;  AL=68H
AAA               ;AL=08 AND CF=0
```

In the example above, ASCII 1 (31H) is added to ASCII 7 (37H). After the AAA instruction, AL will contain 8 in BCD and CF = 0. The following example shows another ASCII addition and then the adjustment:

Example 2:

```
MOV   AL,'9'      ;AL=39 ASCII FOR 9
ADD   AL,'5'      ;ADD 35 (ASCII FOR 5) TO AL THEN AL=6EH
AAA               ;NOW AL=04 CF=1
OR    AL,30H      ;converts result to ASCII
```

AAD ASCII Adjust before Division

Flags: Affected: SF, ZF, PF. Unpredictable: OF, AF, CF.
Format: AAD
Function: Used before the DIV instruction to convert two unpacked BCD digits in AL and AH to binary. A better name for this would be BCD to binary conversion before division. This allows division of ASCII numbers. Before the AAD instruction is executed, the ASCII tag of 3 must be masked from the upper nibble of AH and AL.

Example:

```
MOV   AX,3435H        ;AX=3435 THE ASCII FOR 45
AND   AX,0F0FH        ;AX=0405H UNPACKED BCD FOR 45
AAD                   ;AX=002DH HEX FOR 45
MOV   DL,07           ;DL=07
DIV   DL              ;2DH DIV BY 07 GIVES AL=06,AH=03
OR    AX,3030H        ;AL=36=QUOTIENT AND AH=33=REMAINDER
```

AAM ASCII Adjust after Multiplication

Flags: Affected: AF, CF. Unpredictable: OF, SF, ZF, PF.
Format: AAM
Function: Again, a better name would have been BCD adjust after multiplication. It is used after the MUL instruction has multiplied two unpacked BCD numbers. It converts AX from binary to unpacked BCD. AAM adjusts only AL, and any digits greater than 9 are stored in AH.

Example:

```
MOV   AL,'5'          ;AL=35
AND   AL,0FH          ;AL=05 UNPACKED BCD FOR 5
MOV   BL,'4'          ;BL=34
AND   BL,0FH          ;BL=04 UNPACKED BCD FOR 4
MUL   BL              ;AX=0014H=20 DECIMAL
AAM                   ;AX=0200
OR    AX,3030H        ;AX=3230 ASCII FOR 20
```

AAS ASCII Adjust after Subtraction

Flags: Affected: AF, CF. Unpredictable: OF, SF, ZF, PF.
Format: AAS
Function: After the subtraction of two ASCII digits, this instruction is used to convert the result in AL to packed BCD. Only AL is adjusted; the value in AH will be decremented if the carry flag is set.

Example:

```
MOV   AL,32H          ;AL=32 ASCII FOR 2
MOV   DH,37H          ;DH=37 ASCII FOR 7
SUB   AL,DH           ;AL-DH=32-37=FBH WHICH IS -5 IN 2'S COMP
                      ;CF=1 INDICATING A BORROW
AAS                   ;NOW AL=05 AND CF=1
```

ADC Add with Carry

Flags: Affected: OF, SF, ZF, AF, PF, CF.
Format: ADC dest,source ;dest = dest + source + CF
Function: If CF=1 prior to this instruction, then after execution of this instruction, source is added to destination plus 1. If CF = 0, source is added to destination plus 0. Used widely in multibyte and multiword additions.

ADD Signed or Unsigned ADD

Flags: Affected: OF, SF, ZF, AF, PF, CF.
Format: ADD dest,source ;dest = dest + source
Function: Adds source operand to destination operand and places the result in destination. Both source and destination operands must match (e.g., both byte size or word size) and only one of them can be in memory.

Unsigned addition:

In addition of unsigned numbers, the status of CF, ZF, SF, AF, and PF may change, but only CF, ZF, and AF are of any use to programmers. The most important of these flags is CF. It becomes 1 when there is carry from D7 out in 8-bit (D0 - D7) operations, or a carry from D15 out in 16-bit (D0 - D15) operations.

Example 1:

```
MOV   BH,45H          ;BH=45H
ADD   BH,4FH          ;BH=94H (45H+4FH=94H)
                      ;CF=0,ZF=0,SF=1,AF=1,and PF=0
```

Example 2:

```
MOV   AL,FEH          ;AL=FEH
MOV   DL,75H          ;DL=75H
ADD   AL,DL           ;AL=FE+75=73H
                      ;CF=1,ZF=0,AF=0,SF=0,PF=0
```

Example 3:

```
MOV   DX,126FH        ;DX=126FH
ADD   DX,3465H        ;DX=46D4H (126F=3465=46D4H)
                      ;CF=0,ZF=0,AF=1,SF=0,PF=1
MOV   BX,0FFFFH
ADD   BX,1            ;BX=0000 (FFFFH+1=0000)
                      ;AND CF=1,ZF=1,AF=1,SF=0,PF=1
```

Signed addition:

In addition of signed numbers, the status of OF, ZF, and SF must be noted. Special attention should be given to the overflow flag (OF) since this indicates if there is an error in the result of the addition. There are two rules for setting OF in signed number operation. The overflow flag is set to 1:

1. If there is a carry from D6 to D7 and no carry from D7 out in an 8-bit operation or a carry from D14 to D15 and no carry from D15 out in a 16-bit operation
2. If there is a carry from D7 out and no carry from D6 to D7 in an 8-bit operation or a carry from D15 out but no carry from D14 to D15 in a 16-bit operation

Notice that if there is a carry both from D7 out and from D6 to D7, then OF = 0 in 8-bit operations. In 16-bit operations, OF = 0 if there is both a carry out from D15 and a carry from D14 to D15.

Example 4:

```
MOV   BL,+8           ;BL=0000 1000
MOV   DH,+4           ;DH=0000 0100
ADD   BL,DH           ;BL=0000 1100 SF=0,ZF=0,OF=0,CF=0
```

Notice SF = D7 = 0 since the result is positive and OF = 0 since there is neither a carry from D6 to D7 nor any carry beyond D7. Since OF = 0, the result is correct [(+8) + (+4) = (+12)].

Example 5:

```
MOV   AL,+66          ;AL=0100 0010
MOV   CL,+69          ;CL=0100 0101
ADD   CL,AL           ;CL=1000 0111 = -121 (INCORRECT)
                      ;CF=0,SF=1,ZF=0, AND OF=1
```

In Example 5, the correct result is +135 [(+66) + (+69) = (+135)], but the result was −121. The OF = 1 is an indication of this error. Notice that SF = D7 = 1 since the result is negative; OF = 1 since there is a carry from D6 to D7 and CF=0.

Example 6:

```
MOV   AL,-12        ;AL=1111 0100
MOV   BL,+18        ;BL=0001 0010
ADD   BL,AL         ;BL=0000 0110 (WHICH IS +6 )
                    ;SF=0,ZF=0,OF=0, AND CF=1
```

Notice above that OF = 0 since there is a carry from D6 to D7 and a carry from D7 out.

Example 7:

```
MOV   AH,-30        ;AH=1110 0010
MOV   DL,+14        ;DL=0000 1110
ADD   DL,AH         ;DL=1111 0000 (WHICH IS -16 AND CORRECT)
                    ;AND SF=1,ZF=0,OF=0, AND CF=0
```

OF = 0 since there is no carry from D7 out nor any carry from D6 to D7.

Example 8:

```
MOV   AL,-126       ;AL=1000 0010
MOV   BH,-127       ;BH=1000 0001
ADD   AL,BH         ;AL=0000 0011 (WHICH IS +3 AND WRONG)
                    ;AND SF=0,ZF=0 AND OF=1
```

OF = 1 since there is carry from D7 out but no carry from D6 to D7.

AND Logical AND

Flags: Affected: CF = 0, OF = 0, SF, ZF, PF.
Unpredictable: AF.
Format: AND dest,source
Function: Performs logical AND on the operands, bit by bit, storing the result in the destination.

X Y	X AND Y
0 0	0
0 1	0
1 0	0
1 1	1

Example:

```
MOV   BL,39H ;BL=39
AND   BL,09H ;BL=09
;39 0011 1001
;09 0000 1001
;-  --------
;09 0000 1001
```

CALL Call a Procedure

Flags: Unchanged.
Format: CALL proc ;transfer control to procedure
Function: Transfers control to a procedure. RET is used to return control to the instruction after the call. There are two types of CALLs: NEAR and FAR. If the target address is within the same code segment, it is a NEAR call. If the target address is outside the current code segment, it is a FAR CALL. Each is described below.

NEAR CALL: If calling a near procedure (the procedure is in the same code segment as the CALL instruction) then the content of the IP register (which is the address of the instruction after the CALL) is pushed onto the stack and SP is decremented by 2. Then IP is loaded with the new value, which is the offset of the procedure. At the end of the procedure when the RET is executed, IP is popped off the stack, which returns control to the instruction after the CALL. There are three ways to code the address of the called NEAR procedure:

1. Direct:

```
                CALL    proc1
                ...
proc1           PROC NEAR
                ...
                RET
proc1           ENDP
```

2. Register indirect:

```
                CALL    [SI]        ;transfer control to address in SI
```

3. Memory indirect:

```
                CALL    WORD PTR [DI]  ;DI points to the address that
                                       ;contains IP address of proc
```

FAR CALL: When calling a far procedure (the procedure is in a different segment from the CALL instruction), the SP is decremented by 4 after CS:IP of the instruction following the CALL is pushed onto the stack. CS:IP is then loaded with the segment and offset address of the called procedure. In pushing CS:IP onto the stack, CS is pushed first and then IP. When the RETF is executed, CS and IP are restored from the stack and execution continues with the instruction following the CALL. The following addressing modes are supported:

1. Direct (but outside the present segment):

```
                CALL    proc1
                ...
proc1           PROC FAR
                ...
                RETF
proc1           ENDP
```

2. Memory indirect:

```
                CALL    DWORD PTR [DI]  ;transfer control to CS:IP where
                                        ;DI and DI+1 point to location of CS and
                                        ;DI+2 and DI+4 point to location of IP
```

CBW Convert Byte to Word

Flags: Unchanged.
Format: CBW
Function: Copies D7 (the sign flag) to all bits of AH. Used widely to convert a signed byte in AL into a signed word to avoid the overflow problem in signed number arithmetic.

Example:

```
MOV     AX,0
MOV     AL,-5           ;AL=(-5)=FB in 2's complement
                        ;AX = 0000 0000 1111 1011
CBW                     ;now AX=FFFB
                        ;AX = 1111 1111 1111 1011
```

CLC Clear Carry Flag

Flags: Affected: CF.
Format: CLC
Function: Resets CF to zero (CF = 0).

CLD Clear Direction Flag

Flags: Affected: DF.
Format: CLD
Function: Resets DF to zero (DF = 0). In string instructions if DF = 0, the pointers are incremented with each execution of the instruction. If DF = 1, the pointers are decremented. Therefore, CLD is used before string instructions to make the pointers increment.

CLI Clear Interrupt Flag

Flags: Affected: IF.
Format: CLI
Function: Resets IF to zero, thereby masking external interrupts received on INTR input. Interrupts received on NMI input are not blocked by this instruction.

CMC Complement Carry Flag

Flags: Affected: CF.
Format: CMC
Function: Changes CF from 0 to 1 or from 1 to 0.

CMP Compare Operands

Flags: Affected: OF, SF, ZF, AF, PF, CF.
Format: CMP dest,source ;sets flags as if "SUB dest,source"
Function: Compares two operands of the same size. The source and destination operands are not altered. Performs comparison by subtracting the source operand from the destination and sets flags as if SUB were performed. The relevant flags are as follows:

	CF	ZF	SF	OF
dest > source	0	0	0	SF
dest = source	0	1	0	SF
dest < source	1	0	1	inverse of SF

CMPS/CMPSB/CMPSW Compare Byte or Word String

Flags: Affected: OF, SF, ZF, AF, PF, CF.
Format: CMPSx
Function: Compares strings a byte or word at a time. DS:SI is used to address the first operand; ES:DI is used to address the second. If DF = 0, it increments the pointers SI and DI. If DF = 1, it decrements the pointers. It can be used with prefix REPE or REPNE to compare strings of any length. The comparison is done by subtracting the source operand from the destination and sets flags as if SUB were performed.

CWD Convert Word to Doubleword

Flags: Unchanged.
Format: CWD
Function: Converts a signed word in AX into a signed doubleword by copying the sign bit of AX into all the bits of DX. Often used to avoid the overflow problem in signed number arithmetic.

Example:

```
MOV  DX,0
MOV   AX,-5          ;AX=(-5)=FFFB in 2's complement
;DX = 0000H
CWD
;DX = FFFFH
```

DAA Decimal Adjust after Addition

Flags: Affected: SF, ZF, AF, PF, CF, OF.
Format: DAA
Function: This instruction is used after addition of BCD numbers to convert the result back to BCD. It adds 6 to the lower 4 bits of AL if it is greater than 9 or if AF = 1. Then it adds 6 to the upper 4 bits of AL if it is greater than 9 or if CF = 1.

Example 1:

```
MOV   AL,47H         ;AL=0100 0111
ADD   AL,38H         ;AL=47H+38H=7FH.  invalid BCD
DAA                  ;NOW AL=1000 0101 (85H IS VALID BCD)
```

In this example, since the lower nibble was larger than 9, DAA added 6 to AL. If the lower nibble is smaller than 9 but AF = 1, it also adds 6 to the lower nibble.

Example 2:

```
MOV   AL,29H         ;AL=0010 1001
ADD   AL,18H         ;AL=0100 0001 INCORRECT RESULT
DAA                  ;AL=0100 0111 A VALID BCD FOR 47H.
```

The same thing can happen for the upper nibble.

Example 3:

```
MOV   AL,52H         ;AL=0101 0010
ADD   AL,91H         ;AL=1110 0011 AN INVALID BCD
DAA                  ;AL=0100 0011 AND CF=1
```

Again the upper nibble can be smaller than 9 but because CF = 1, it must be corrected.

Example 4:

```
MOV   AL,94H         ;AL=1001 0100
ADD   AL,91H         ;AL=0010 0101 INCORRECT RESULT
DAA                  ;AL=1000 0101 A VALID BCD FOR 85 AND CF=1
```

It is entirely possible that 6 is added to both the high and low nibbles.

Example 5:

```
MOV   AL,54H         ;AL=0101 0100
ADD   AL,87H         ;AL=1101 1011 INVALID BCD
DAA                  ;AL=0100 0001 AND CF=1 (141 IN BCD)
```

DAS Decimal Adjust after Subtraction

Flags: Affected: SF, ZF, AF, PF, CF. Unpredictable: OF.
Format: DAS
Function: This instruction is used after subtraction of BCD numbers to convert the result to BCD. If the lower 4 bits of AL represent a number greater than 9 or if AF = 1, then 6 is subtracted from the lower nibble. If the upper 4 bits of AL is now greater than 9 or if CF = 1, 6 is subtracted from the upper nibble.

Example:

```
MOV   AL,45H      ;AL=0100 0101 BCD for 45
SUB   AL,17H      ;AL=0010 1110 AN INVALID BCD
DAS               ;AL=0010 1000 BCD FOR 28(45-17=28)
```

For more examples of problems associated with BCD arithmetic, see DAA.

DEC Decrement

Flags: Affected: OF, SF, ZF, AF, PF. Unchanged: CF.
Format: DEC dest ;dest = dest - 1
Function: Subtracts 1 from the destination operand. Note that CF (carry/borrow) is unchanged even if a value 0000 is decremented and becomes FFFF.

DIV Unsigned Division

Flags: Unpredictable: OF, SF, ZF, AF, PF, CF.
Format: DIV source ;divide AX or DX:AX by source
Function: Divides either an unsigned word (AX) by a byte or an unsigned doubleword (DX:AX) by a word. If dividing a word by a byte, the quotient will be in AL and the remainder in AH. If dividing a doubleword by a word, the quotient will be in AX and the remainder in DX. Divide by zero causes interrupt type 0.

ESC Escape

Flags: Unchanged.
Format: ESC
Function: This instruction facilitates the use of math coprocessors (such as the 8087), which share data and address buses with the microprocessor. ESC is used to pass an instruction to a coprocessor and is usually treated as NOP (no operation) by the main processor.

HLT Halt

Flags: Unchanged.
Format: HLT
Function: Causes the microprocessor to halt execution of instructions. To get out of the halt state, activate an interrupt (NMI or INTR) or RESET.

IDIV Signed Number Division

Flags: Unpredictable: OF, SF, ZF, AF, PF, CF.
Format: IDIV source ;divide AX or DX:AX by source
Function: This division function divides either a signed word (AX) by a byte or a signed doubleword (DX:AX) by a word. If dividing a word by a byte, the signed quotient will be in AL and the signed remainder in AH. If dividing a doubleword by a word, the signed quotient will be in AX and the signed remainder in DX. Divide by zero causes interrupt type 0.

IMUL Signed Number Multiplication

Flags: Affected: OF, CF. Unpredictable: SF, ZF, AF, PF.
Format: IMUL source ;AX =source x AL or DX:AX =source x AX
Function: Multiplies a signed byte or word source operand by a signed byte or word in AL or AX with the result placed in AX or DX:AX.

IN Input Data from Port

Flags: Unchanged.
Format: IN accumulator,port ;input byte or word into AL or AX
Function: Transfers a byte or word to AL or AX from an input port specified by the second operand. The port address can be direct or register indirect:

1. Direct: port address is specified directly and cannot be larger than FFH.

Example 1:

```
IN  AL,99H          ;BRING A BYTE INTO AL FROM PORT 99H
```

Example 2:

```
IN  AX,78H          ;BRING A WORD FROM PORT ADDRESSES 78H
                    ;AND 79H.  THE BYTE FROM PORT 78 GOES
                    ;TO AL AND BYTE FROM PORT 79H TO AH.
```

2. Register indirect: the port address is kept by the DX register. Therefore, it can be as high as FFFFH.

Example 3:

```
MOV   DX,481H       ;DX=481H
IN    AL,DX         ;BRING THE BYTE TO AL FROM THE PORT
                    ;WHOSE ADDRESS IS POINTED BY DX
```

Example 4:

```
IN    AX,DX         ;BRING A WORD FROM PORT ADDRESS OF
                    ;POINTED BY DX.  THE BYTE FROM PORT
                    ;DX GOES TO AL AND BYTE FROM PORT
                    ;DX+1 TO AH.
```

INC Increment

Flags: Affected: OF, SF, ZF, AF, PF. Unchanged: CF.
Format: INC destination ;dest = dest + 1
Function: Adds 1 to the register or memory location specified by the operand. Note that CF is not affected even if a value FFFF is incremented to 0000.

INT Interrupt

Flags: Affected: IF, TF.
Format: INT type ;transfer control to INT type
Function: Transfers execution to one of the 256 interrupts. The vector address is specified by the type number, which cannot be greater than FFH (0 to FF = 256 interrupts).

The following steps are performed for the interrupt:

1. SP is decremented by 2 and the flags are pushed onto the stack.
2. SP is decremented by 2 and CS is pushed onto the stack.
3. SP is decremented by 2 and the IP of the next instruction after the interrupt is pushed onto the stack.
4. Multiplies the type number by 4 to get the address of the vector table. Starting at this address, the first 2 bytes are the value of IP and the next 2 bytes are the value for CS of the interrupt handler (interrupt handler is also called interrupt service routine).
5. Resets IF and TF.

Interrupts are used to get the attention of the microprocessor. In the 8086/88 there are a total of 256 interrupts: INT 00, INT 01, INT 02, ... , INT FF. As mentioned above, the address that an interrupt jumps to is always four times the value of the interrupt number. For example, INT 03 will jump to memory address 0000CH (4 × 03 = 12 = 0CH). Table B-1 is a partial list of the interrupts, commonly referred to as the interrupt vector table.

Table B-1: Interrupt Vector Table

INT # (hex)	Physical Address	Logical Address
INT 00	00000	0000:0000
INT 01	00004	0000:0004
INT 02	00008	0000:0008
INT 03	0000C	0000:000C
INT 04	00010	0000:0010
INT 05	00014	0000:0014
...	...	...
INT FF	003FC	0000:03FC

Every interrupt has a program associated with it called the interrupt service routine (ISR). When an interrupt is invoked, the CS:IP address of its ISR is retrieved from the vector table (shown above). The lowest 1024 bytes (256 × 4 = 1024) of RAM are set aside for the interrupt vector table and must not be used for any other function.

Example: Find the physical and logical addresses of the vector table associated with (a) INT 14H and (b) INT 38H.

Solution:

(a) The physical address for INT 14H is 00050H - 00053H

(4 × 14H = 50H). That gives the logical address of 0000:0050H - 0000:0053H.

(b) The physical address for INT 38H is 000E0H - 000E3H, making the physical address 0000:00E0H - 0000:00E3H.

The difference between INTerrupt and CALL instructions

The following are some of the differences between the INT and CALL FAR instructions:

1. While a CALL can jump to any location within the 1-megabyte address range (00000 - FFFFF) of the 8088/86 CPU, "INT nn" jumps to a fixed location in the vector table as discussed earlier.
2. While the CALL is used by the programmer at a predetermined point in a program, a hardware interrupt can come in at any time.
3. A CALL cannot be masked (disabled), but "INT nn" can be masked.
4. While a "CALL FAR" automatically saves on the stack only the CS:IP of the next instruction, "INT nn" saves the FR (flag register) in addition to the CS:IP.
5. While at the end of the procedure that has been CALLed the RETF (return FAR) is used, for "INT nn" the instruction IRET (interrupt return) is used.

The 256 interrupts can be categorized into two different groups: hardware and software interrupts.

Hardware interrupts

The 8086/88 microprocessors have two pins set aside for inputting hardware interrupts. They are INTR (interrupt request) and NMI (nonmaskable interrupt). Although INTR can be ignored through the use of software masking, NMI cannot be masked using software. These interrupts are activated externally by putting 5 volts on the hardware pins of NMI or INTR. Intel has assigned INT 02 to NMI. When it is activated it will jump to memory location 00008 to get the address (CS:IP) of the interrupt service routine (ISR). Memory locations 00008, 00009, 0000A, and 0000B contain the 4-byte CS:IP. There is no specific location in the vector table assigned to INTR because INTR is used to expand the number of hardware interrupts and should be allowed to use any "INT nn" instruction that has not been assigned previously. In the IBM PC, one Intel 8259 PIC (programmable interrupt controller) chip is connected to INTR to add a total of eight hardware interrupts to the microprocessor. IBM PC AT, PS/2 80286, 80386, and 80486 computers use two 8259 chips to allow up to 15 hardware interrupts.

Table B-2: IBM PC Interrupt System

Interrupt	Logical Address	Physical Address	Purpose
0	00E3:3072	03EA2	Divide error
1	0600:08ED	068ED	Single step (trace command in DEBUG)
2	F000:E2C3	FE2C3	Nonmaskable interrupt
3	0600:08E6	068E6	Breakpoint
4	0700:0147	00847	Signed number arithmetic overflow
5	F000:FF54	FFF54	Print screen (BIOS)
10	F000:F065	FF065	Video I/O (BIOS)
...	...	...	...
21	relocatable	---	DOS function calls
...	...	...	...

Software interrupts

These interrupts are called software interrupts since they are invoked as a result of the execution of an instruction and no external hardware is involved. In other words, these interrupts are invoked by executing an "INT nn" instruction such as the DOS function call "INT 21H" or video interrupt "INT 10H". These interrupts can be invoked by a program at any time, the same as any other instruction. Many of the interrupts in this category are used by the DOS operating system and IBM BIOS to perform the essential tasks that every computer must provide to the system and the user. Also within this group of interrupts are predefined functions associated with some of the interrupts. They are "INT 00" (divide error), "INT 01" (single step), "INT 03" (breakpoint), and "INT 04" (signed number overflow). Each one is described below. These interrupts are shown in Table B-2. Looking at Table B-2, one can say that aside from "INT 00" to "INT 04", which have predefined functions, the rest of the interrupts, from "INT 05" to "INT FF", can be used to implement either software or hardware interrupts.

Functions associated with "INT 00" to "INT 03"

As mentioned earlier, interrupts "INT 00" to "INT 03" have predefined functions and cannot be used in any other way. The function of each is described next.

INT 00 (divide error)

This interrupt, sometimes referred to as a conditional or exception interrupt, is invoked by the microprocessor whenever there is a condition that it cannot take care of, such as an attempt to divide a number by zero. "INT 00" is invoked by the microprocessor whenever there is an attempt to divide a number by zero. In the IBM PC and compatibles, the service subroutine for this interrupt is responsible for displaying the message "DIVIDE ERROR" on the screen if a program such as the following is executed:

```
MOV   AL,25   ; put 25 into AL
MOV   BL,00   ; put 00 into BL
DIV   BL      ; divide 25 by 00
```

This interrupt is also invoked if the quotient is too large to fit into the assigned register when executing a DIV instruction.

INT 01 (single step)

There is often a need to execute a given program one instruction at a time and then inspect the registers (possibly memory as well) to see what is happening inside the CPU. This is commonly referred to as single-stepping. IBM and Microsoft call it TRACE in the DEBUG program. To allow the implementation of single-stepping, Intel has set aside "INT 01" specifically for that purpose. For the Trace command in DEBUG after execution of each instruction, the CPU jumps automatically to physical location 00004 to fetch the 4 bytes for CS:IP of the interrupt service routine. One of the functions of this ISR is to dump the contents of the registers onto the screen.

INT 02 (nonmaskable interrupt)

This interrupt is used in the PC to indicate memory errors, among other problems.

INT 03 (breakpoint)

While in single-step mode, one can inspect the CPU and system memory after execution of each instruction. A breakpoint allows one to do the same thing, after execution of a group of instructions rather than after each instruction. Breakpoints are put in at certain points of a program to monitor the flow of the program and to inspect the results after certain instructions. The CPU executes the program to the breakpoint and stops. One can proceed from breakpoint to breakpoint until the program is complete. With the help of single-step and breakpoints, programs can be debugged and tested more easily. The Intel 8086/88 CPUs have set aside "INT 03" for the sole purpose of implementing breakpoints. When the instruction "INT 03" is placed in a program the CPU will execute the program until it encounters "INT 03", and then it stops. One interesting point about this interrupt is that it is a one-byte instruction, in contrast to all other interrupt instructions, "INT nn", which are two-byte instructions. This allows the user to insert 1 byte of code and remove it to proceed with the execution of the program. The opcode for INT 03 is "CC".

IBM PC and DOS assignment of interrupts

When the IBM PC was being developed, the designers at IBM had to coordinate the assignment of the 256 available interrupts for the 8086/88 with Microsoft, the developer of the DOS operating system, lest a conflict occur between the BIOS and DOS interrupt designations. The result of cooperation in assigning interrupts to IBM BIOS subroutines and DOS function calls is shown in Table B-2. The table gives a partial listing of interrupt numbers from 00 to FF, the logical address of the service subroutine for each interrupt, their physical addresses, and the purpose of each interrupt. It must be mentioned that depending on the computer and the DOS version, some of the logical addresses could be different from Table B-2.

How to get the vector table of any PC

One can get the vector table of any IBM PC/XT, PC AT, PS/2, PS/1, or any 80x86 IBM-compatible computer and inspect the logical address assigned to each interrupt. To do that use DEBUG's DUMP command "-D 0000:0000", as shown next.

```
A>debug
-D 0000:0000
0000:0000  E8 56 2B 02 56 07 70 00-C3 E2 00 F0 56 07 70 00  .....
0000:0010  56 07 70 00 54 FF 00 F0-47 FF 00 F0 47 FF 00 F0  .....
```

Note: The contents of the memory locations could be different, depending on the DOS version.

Example: From the dump above, find the CS:IP of the service routine associated with INT 5.

Solution: To get the address of "INT 5", calculate the physical address of 00014H (5 × 4 = 00014H). The contents of these locations are 00014 = 54, 00015 = FF, 00016 = 00, and 00016 = F0. This gives CS = F000 and IP = FF54.

INTO Interrupt on Overflow

Flags: Affected: IF, TF.
Format: INTO
Function: Transfers execution to an interrupt handler written for overflow if OF (overflow flag) has been set. Intel has set aside INT 4 for this purpose. Therefore, if OF = 1 when INTO is executed, the CPU jumps to memory location 00010H (4 × 4 = 16 = 10H). The contents of memory locations 10H, 11H, 12H, and 13H are used as IP and CS of the interrupt handler procedure. This instruction is widely used to detect overflow in signed number addition. In signed number operations, OF becomes 1 in two cases:

1. Whenever there is a carry from d6 to d7 in 8-bit operations and no carry from D7 out (or in 16-bit operations when there is carry from d14 to d15 and CF = 0)
2. When there is carry from from D7 out and no carry from D6 to D7 (or in the case of 16-bit operation when there is a carry from D15 out and no carry from D14 to D15)

IRET Interrupt Return

Flags: Affected: OF, DF, IF, TF, SF, ZF, AF, PF, CF.
Format: IRET
Function: Used at the end of an interrupt service routine (interrupt handler), this instruction restores all flags, CS, and IP to the values they had before the interrupt so that execution may continue at the next instruction following the INT instruction. While the RET instruction is used at the end of the subroutine associated

with the CALL instruction, IRET must be used for the subroutine associated with the "INT XX" instruction or the hardware interrupt handler.

JUMP Instructions

The following instructions are associated with jumps (both conditional and unconditional). They are categorized according to their usage rather than alphabetically.

J condition

Flags: Unchanged.
Format: Jxx target ;jump to target upon condition
Function: Used to jump to a target address if certain conditions are met. The target address cannot be more than −128 to +127 bytes away. The conditions are indicated by the flag register. The conditions that determine whether the jump takes place can be categorized into three groups,

(1) flag values,
(2) the comparison of unsigned numbers, and
(3) the comparison of signed numbers.
Each is explained next.

1. "J condition" where the condition refers to flag values. The status of each bit of the flag register has been decided by execution of instructions prior to the jump. The following "J condition" instructions check if a certain flag bit is raised or not.

JC	Jump Carry	jump if CF=1
JNC	Jump No Carry	jump if CF=0
JP	Jump Parity	jump if PF=1
JNP	Jump No Parity	jump if PF=0
JZ	Jump Zero	jump if ZF=1
JNZ	Jump No Zero	jump if ZF=0
JS	Jump Sign	jump if SF=1
JNS	Jump No Sign	jump if SF=0
JO	Jump Overflow	jump if OF=1
JNO	Jump No Overflow	jump if OF=0

Notice that there is no "J condition" instruction for AF.

2. "J condition" where the condition refers to the comparison of unsigned numbers. After a compare (CMP dest,source) instruction is executed, CF and ZF indicate the result of the comparison, as follows:

	CF	ZF
destination > source	0	0
destination = source	0	1
destination < source	1	0

Since the operands compared are viewed as unsigned numbers, the following "J condition" instructions are used.

JA	Jump Above	jump if CF=0 and ZF=0
JAE	Jump Above or Equal	jump if CF=0
JB	Jump Below	jump if CF=1
JBE	Jump Below or Equal	jump if CF=1 or ZF=1
JE	Jump Equal	jump if ZF=1
JNE	Jump Not Equal	jump if ZF=0

3. "J condition" where the condition refers to the comparison of signed numbers. In the case of the signed number comparison, although the same instruction, "CMP destination,source", is used, the flags used to check the result are as follows:

destination > source	OF=SF or ZF=0
destination = source	ZF=1
destination < source	OF inverse of SF

Consequently, the "J condition" instructions used are different. They are as follows:

JG	Jump Greater	jump if ZF=0 or OF=SF
JGE	Jump Greater or Equal	jump if OF=SF
JL	Jump Less	jump if OF≠SF
JLE	Jump Less or Equal	jump if ZF=1 or OF≠SF
JE	Jump if Equal	jump if ZF = 1

There is one more "J condition" instruction:

```
JCXZ        ;Jump if CX is Zero.  ZF is ignored.
```

All "J condition" instructions are short jumps, meaning that the target address cannot be more than −128 bytes backward or +127 bytes forward from the IP of the instruction following the jump. What happens if a programmer needs to use a "J condition" to go to a target address beyond the −128 to +127 range? The solution is to use the "J condition" along with the unconditional JMP instruction, as shown next.

```
            ADD     BX,[SI]
            JNC     NEXT
            JMP     TARGET1
NEXT:       ....
            ...
TARGET1:    ADD     DI,10
            ...
```

JMP Unconditional Jump

Flags: Unchanged.

Format: JMP [directives] target ;jump to target address

Function: This instruction is used to transfer control unconditionally to a new address. The difference between JMP and CALL is that the CALL instruction will return and continue execution with the instruction following the CALL, whereas JMP will not return. The target address could be within the current code segment, which is called a near jump, or outside the current code segment, which is called a far jump. Within each category there are many ways to code the target address, as shown next.

1. Near jump

(a) direct short jump: In this jump the target address must be within −128 to +127 bytes of the IP of the instruction after the JMP. This is a 2-byte instruction. The first byte is the opcode EBH and the second byte is the signed number displacement, which is added to the IP of the instruction following the JMP to get the target address. The directive SHORT must be coded, as shown next:

```
            JMP  SHORT OVER
            ...
OVER:       ...
```

If the target address is beyond the −128 to +127 byte range and the SHORT directive is coded, the assembler gives an error.

(b) Direct jump: This is a 3-byte instruction. The first byte is the opcode E9H and the next two bytes are the signed number displacement value. The displacement is added to the IP of the instruction following the JMP to get the target address. The displacement can be in the range −32,768 to +32,767. In the absence of the SHORT directive, the assembler in its first pass always uses this kind of JMP, and then in the second pass if the target address is within the −128 and +127 byte range, it uses the NOP opcode 90H for the third byte. This is the reason to code the directive SHORT if it is known that the target address of the JMP is within the short range.

(c) Register indirect jump: In this jump the target address is in a register as shown next:

```
JMP    DI  ;jump to the address found in DI
```

Any nonsegment register can be used for this purpose.

(d) Memory indirect jump: In this jump the target address is in a memory location whose address is pointed at by a register:

```
JMP    WORD PTR [SI] ;jump to the address found at the address in SI
```

The directive WORD PTR must be coded to indicate this is a near jump.

2. Far jump

In a far JMP, the target address is outside the present code segment; therefore, not only the offset value but also the segment value of the target address must be given. A far jump is a 5-byte instruction: the opcode EAH and 4 bytes for the offset and segment of the target address. The following shows the two methods of coding the far jump.

(a) Direct far jump: This requires that both CS and IP be updated. One way to do that is to use the LABEL directive:

```
          JMP       TARGET2
          ...
TARGET2   LABEL     FAR
ENTRY:    ...
```

This is exactly what IBM has done in BIOS of the IBM PC/XT when the computer is booted. When the power to the PC is turned on, the 8088/86 CPU begins to execute at address FFFF:0000H. IBM uses a FAR jump to make it go to location F000:E05BH, as shown next:

```
                          ;CS=FFFF and IP=0000
0000 EA5BE000F0           JMP    RESET
                          ;CS=F000
                          ORG    0E05BH
E05B   RESET              LABEL  FAR
E05B   START:
E05B                      CLI
E05C                      ...
```

The EXTRN and PUBLIC directives also can be used for the same purpose.

(b) Memory indirect far jump: The target address (both CS:IP) is in a memory location pointed to by the register:

```
JMP    DWORD PTR [BX]
```

The DWORD and PTR directives must be used to indicate that it is a far jump.

LAHF Load AH from Flags

Flags: Unchanged.
Format: LAHF
Function: Loads the lower 8 bits of the flag register into AH.

LDS Load Data Segment Register

Flags: Unchanged.
Format: LDS dest,source ;load dest and DS starting at source
Function: Loads into destination (which is a register) the contents of two memory locations indicated by source and loads DS with the contents of the two succeeding memory locations. This is useful for accessing a new data segment and its offset.

Example: Assume the following memory locations with the contents:

```
;DS:1200=(46)
;DS:1201=(10)
;DS:1202=(38)
;DS:1203=(82)
LDS    DI,[1200]          ;now DI=1046 and DS=8238.
```

EA Load Effective Address

Flags: Unchanged.
Format: LEA dest,source ;dest = OFFSET source
Function: Loads into the destination (a 16-bit register) the effective address of a direct memory operand.

Example 1:

```
       ORG    0100H
DATA   DB     34,56,87,90,76,54,13,29
       ...
       ;to access the sixth element:
       LEA    SI,DATA+5      ;SI=100H+5=105 THE EFFECTIVE ADDRESS
       MOV    AL,[SI]        ;GET THE SIXTH ELEMENT
```

Example 2:

```
       ;if BX=2000H and SI=3500H
       LEA    DX,[BX][SI]+100H
       ;DX=effective address=2000+3500+100=5600H
```

The following two instructions show two different ways to accomplish the same thing:

```
       MOV    SI,OFFSET DATA      ;advantage: executes faster
       LEA    SI,DATA
```

LES Load Extra Segment Register

Flags: Unchanged.
Format: LES dest,source ;load dest and ES starting at source
Function: Loads into destination (a register) the contents of two memory locations indicated by the source and loads ES with the contents of the two succeeding memory locations. Useful for accessing a new extra segment and its offset. This instruction is similar to LDS except that the ES and its offset are being loaded.

LOCK Lock System Bus Prefix

Flags: Unchanged.
Format: LOCK ;used as a prefix before instructions
Function: Used in microcomputer systems with more than one processor to prevent another processor from gaining control over the system bus during execution of an instruction.

LODS/LODSB/LODSW Load Byte or Word String

Flags: Unchanged.
Format: LODSx
Function: Loads AL or AX with a byte or word from the memory location pointed to by DS:SI. If DF = 0, SI will be incremented to point to the next location. If DF = 1, SI will be decremented to point to the next location. SI is incremented/decremented by 1 or 2, depending on whether it is a byte or word string.

LOOP Loop until CX=0

Flags: Unchanged.
Format: LOOP target ;DEC CX, then jump to target if CX not 0
Function: Decrements CX by 1, then jumps to the offset indicated by the operand if CX not zero, otherwise continues with the next instruction below the LOOP. This instruction is equivalent to

```
DEC CX
JNZ target
```

LOOPE/LOOPZ LOOP if Equal / Loop if Zero

Flags: Unchanged.
Format: LOOPx target ;DEC CX, jump to target if CX $\neq$ 0 and ZF=1
Function: Decrements CX by 1, then jumps to location indicated by the operand if CX is not zero and ZF is 1, otherwise continues with the next instruction after the LOOP. In other words, it gets out of the loop only when CX becomes zero or when ZF = 0.

Example:
Assume that 200H memory locations from offset 1680H should contain 55H. LOOPE can be used to see if any of these locations does not contain 55H:

```
        MOV   CX,200        ;SET UP THE COUNTER
        MOV   SI,1680H      ;SET UP THE POINTER
BACK:   CMP   [SI],55H      ;COMPARE THE 55H WITH MEM LOCATION
                            ;POINTED AT BY SI
        INC   SI            ;INCREMENT THE POINTER
        LOOPE BACK          ;CONTINUE THE PROCESS UNTIL CX=0 OR
                            ;ZF=0. IN OTHER WORDS EXIT IF ONE
                            ;LOCATION DOES NOT HAVE 55H
```

LOOPNE/LOOPNZ LOOP While CF Not Zero and ZF Equal Zero

Flags: Unchanged.
Format: LOOPxx target ;DEC CX, then jump if CX and ZF not zero
Function: Decrements CX by 1, then jumps to location indicated by the operand if CX and ZF are not zero, otherwise continues with the next instruction below the LOOP. In other words it will exit the loop if CX becomes 0 or ZF = 1.

Example:
Assume that the daily temperatures for the last 30 days have been stored starting at memory location with offset 1200H. LOOPE can be used to find the first day that had a 90-degree temperature.

```
        MOV     CX,30       ;SET UP THE COUNTER
        MOV     DI,1200H    ;SET UP THE POINTER
AGAIN:  CMP     [DI],90
        INC     DI
        LOOPNE  AGAIN
```

MOV Move

Flags: Unchanged.
Format: MOV dest,source ;copy source to dest
Function: Copies a word or byte from a register, memory location, or immediate number to a register or memory location. Source and destination must be of the same size and cannot both be memory locations.

MOVS/MOVSB/MOVSW Move Byte or Word String

Flags: Unchanged.
Format: MOVSx
Function: Moves byte or word from memory location pointed to by DS:SI to memory location pointed to by ES:DI. If DF = 0, both pointers are incremented; otherwise, they are decremented. SI and DI are incremented/decremented by 1 or 2 depending on whether it is a byte or word string. When used with the REP prefix, CX is decremented each time until CX is zero.

MUL Unsigned Multiplication

Flags: Affected: OF, CF. Unpredictable: SF, ZF, AF, PF.
Format: MUL source ;AX = source × AL or DX:AX = source × AX
Function: Multiplies an unsigned byte or word indicated by the operand by a unsigned byte or word in AL or AX with the result placed in AX or DX:AX.

NEG Negate

Flags: Affected: OF, SF, ZF, AF, PF, CF.
Format: NEG dest ;negates operand
Function: Performs 2's complement of operand. Effectively reverses the sign bit of the operand. This instruction should only be used on signed numbers.

NOP No Operation

Flags: Unchanged.
Format: NOP
Function: Performs no operation. Sometimes used for timing delays to waste clock cycles. Updates IP to point to next instruction following NOP.

NOT Logical NOT

Flags: Unchanged.
Format: NOT dest ;dest = 1's complement of dest
Function: Replaces the operand with its negation (the 1's complement). Each bit is inverted.

OR Logical OR

Flags: Affected: CF=0, OF=0, SF, ZF, PF. Unpredictable: AF.
Format: OR dest,source ;dest= dest OR source
Function: Performs logical OR on the bits of two operands, replacing the destination operand with the result. Often used to turn a bit on.

X Y	X OR Y
0 0	0
0 1	1
1 0	1
1 1	1

OUT Output Byte or Word

Flags: Unchanged.
Format: OUT dest,acc ;transfer acc to port dest
Function: Transfers a byte or word from AL or AX to an output port specified by the first operand. Port address can be direct or register indirect as shown next:

1. Direct: port address is specified directly and cannot be larger than FFH.

Example 1:

```
OUT    68H,AL          ;SEND OUT A BYTE FROM AL TO PORT 68H
or
OUT    34H,AX ;SEND OUT A WORD FROM AX TO PORT
                       ;ADDRESSES 34H AND 35H. THE BYTE
                       ;FROM AL GOES TO PORT 34H AND
                       ;THE BYTE FROM AH GOES TO PORT 35H
```

2. Register indirect: port address is kept by the DX register. Therefore, it can be as high as FFFFH.

Example 2:

```
MOV    DX,64B1H        ;DX=64B1H
OUT    DX,AL           ;SENT OUT THE BYTE IN AL TO THE PORT
                       ;WHOSE ADDRESS IS POINTED TO BY DX
or
OUT    DX,AX           ;SEND OUT A WORD FROM AX TO PORT
                       ;ADDRESS POINTED TO DX.  THE BYTE
                       ;FROM AL GOES TO PORT DX AND AND BYTE
                       ;FROM AH GOES TO PORT DX+1.
```

POP POP Word

Flags: Unchanged.
Format: POP dest ;dest = word off top of stack
Function: Copies the word pointed to by the stack pointer to the register or memory location indicated by the operand and increments the SP by 2.

POPF POP Flags off Stack

Flags: OF, DF, IF, TF, SF, ZF, AF, PF, CF.
Format: POPF
Function: Copies bits previously pushed onto the stack with the PUSHF instruction into the flag register. The stack pointer is then incremented by 2.

PUSH PUSH Word

Flags: Unchanged.
Format: PUSH source ;PUSH source onto stack
Function: Copies the source word to the stack and decrements SP by 2.

PUSHF PUSH Flags onto stack

Flags: Unchanged.
Format: PUSHF
Function: Decrements SP by 2 and copies the contents of the flag register to the stack.

RCL/RCR Rotate Left through Carry and Rotate Right through Carry

Flags: Affected: OF, CF.
Format: RCx dest,n ;dest = dest rotate right/left n bit positions
Function: Rotates the bits of the operand right or left. The bits rotated out of the operand are rotated into the CF and the CF is rotated into the opposite end of the word or byte. *Note*: "n" must be 1 or CL.

RET Return from a Procedure

Flags: Unchanged.
Format: RET [n] ;return from procedure
Function: Used to return from a procedure previously entered by a CALL instruction. The IP is restored from the stack and the SP is incremented by 2. If the procedure was FAR, then RETF (return FAR) is used, and in addition to restoring the IP, the CS is restored from the stack and SP is again incremented by 2. The RET instruction may be followed by a number that will be added to the SP after the SP has been incremented. This is done to skip over any parameters being passed back to the calling program segment.

ROL/ROR Rotate Left and Rotate Right

Flags: Affected: OF, CF.
Format: ROx dest,n ;rotate dest right/left n bit positions
Function: Rotates the bits of a word or byte indicated by the second operand right or left. The bits rotated out of the word or byte are rotated back into the word or byte at the opposite end. *Note*: "n" must be 1 or CL.

SAHF Store AH in Flag Register

Flags: Affected: SF, ZF, AF, PF, CF.
Format: SAHF
Function: Copies AH to the lower 8 bits of the flag register.

SAL/SAR Shift Arithmetic Left/ Shift Arithmetic Right

Flags: Affected: OF, SF, ZF, PF, CF. Unpredictable: AF.
Format: SAx dest,n ;shift signed dest left/right n bit positions
Function: Shifts a word or byte left /right. SAR/ SAL arithmetic shifts are used for signed number shifting. In SAL, as the operand is shifted left bit by bit, the LSB is filled with 0s and the MSB is copied to CF. In SAR, as each bit is shifted right, the LSB is copied to CF and the empty bits filled with the sign bit (the MSB). SAL/SAR essentially multiply/divide destination by a power of 2 for each bit shift. *Note*: "n" must be 1 or CL.

SBB Subtract with Borrow

Flags: Affected: OF, SF, ZF, AF, PF, CF.
Format: SBB dest,source ;dest = dest - CF - source
Function: Subtracts source operand from destination, replacing destination. If CF =1, it subtracts 1 from the result; otherwise, it executes like SUB.

SCAS/SCASB/SCASW Scan Byte or Word String

Flags: Affected: OF, SF, ZF, AF, PF, CF.
Format: SCASx
Function: Scans a string of data pointed by ES:DI for a value that is in AL or AX. Often used with the REPE/REPNE prefix. If DF is zero, the address is incremented; otherwise, it is decremented.

SHL/SHR Shift Left/Shift Right

Flags: Affected: OF, SF, ZF, PF, CF. Unpredictable: AF.
Format: SHx dest,n ;shift unsigned dest left/right n bit positions
Function: These are logical shifts used for unsigned numbers, meaning that the sign bit is treated as data. In SHR, as the operand is shifted right bit by bit and copied into CF, the empty bits are filled with 0s instead of the sign bit as is the case for SAR. In the case of SHL, as the bits are shifted left, the MSB is copied to CF and empty bits are filled with 0, which is exactly the same as SAL. In reality, SAL and SHL are two different mnemonics for the same opcode. SHL/SHR essentially multiply/divide the destination by a power of 2 for each bit position shifted. *Note*: "n" must be 1 or CL.

STC Set Carry Flag

Flags: Affected: CF.
Format: STC
Function: Sets CF to 1.

STD Set Direction Flag

Flags: Affected: DF.
Format: STD
Function: Sets DF to 1. Used widely with string instructions. As explained in the string instructions, if DF = 1, the pointers are decremented.

STI Set Interrupt Flag

Flags: Affected: IF.
Format: STI
Function: Sets IF to 1, allowing the hardware interrupt to be recognized through the INTR pin of the CPU.

STOS/STOSB/STOSW Store Byte or Word String

Flags: Unchanged.
Format: STOSx
Function: Copies a byte or word from AX or AL to a location pointed to by ES:DI and updates DI to point to the next string element. The pointer DI is incremented if DF is zero; otherwise, it is decremented.

SUB Subtract

Flags: Affected: OF, SF, ZF, AF, PF, CF.
Format: SUB dest,source ;dest = dest - source
Function: Subtracts source from destination and puts the result in the destination. Sets the carry and zero flag according to the following:

	CF	ZF	
dest >source	0	0	the result is positive
dest=source	0	1	the result is 0
dest < source	1	0	the result is negative in 2's comp

The steps for subtraction performed by the internal hardware of the CPU are as follows:

1. Takes the 2's complement of the source
2. Adds this to the destination
3. Inverts the carry and changes the flags accordingly

The source operand remains unchanged by this instruction.

TEST Test Bits

Flags: Affected: OF, SF, ZF, PF, CF. Unpredictable: AF.
Format: TEST dest,source ;performs dest AND source
Function: Performs a logical AND on two operands, setting flags but leaving the contents of both source and destination unchanged. While the AND instruction changes the contents of the destination and the flag bits, the TEST instruction changes only the flag bits.

Example:
Assume that D0 and D1 of port 27 indicate conditions A and B, respectively, if they are high and only one of them can be high at a given time. The TEST instruction can be used as follows:

```
        IN    AL,PORT_27
        TEST AL,0000 0001B      ;CHECK THE CONDITION A
        JNZ   CASE_A            ;JUMP TO INDICATE CONDITION A
        TEST AL,0000 0010B      ;CHECK FOR CONDITION B
        JNZ   CASE_B            ;JUMP TO INDICATE CONDITION B
        ....                    ;THERE IS AN ERROR SINCE NEITHER
        ....                    ;   A OR B HAS OCCURRED.
CASE_A: ....
        ....
CASE_B: ....
```

WAIT Puts Processor in WAIT State

Flags: Unchanged.
Format: WAIT
Function: Causes the microprocessor to enter an idle state until an external interrupt occurs. This is often done to synchronize it with another processor or with an external device.

XCHG Exchange

Flags: Unchanged.
Format: XCHG dest,source ;swaps dest and source
Function: Exchanges the contents of two registers or a register and a memory location.

XLAT Translate

Flags: Unchanged.
Format: XLAT
Function: Replaces contents of AL with the contents of a look-up table whose address is specified by AL. BX must be loaded with the start address of the look-up table and the element to be translated must be in AL prior to the execution of this instruction. AL is used as an offset within the conversion table. Often used to translate data from one format to another, such as ASCII to EBCDIC.

XOR Exclusive OR

Flags: Affected: CF = 0, OF = 0, SF, ZF, PF. Unpredictable: AF.
Format: XOR dest,source
Function: Performs a logical exclusive OR on the bits of two operands and puts the result in the destination. "XOR AX,AX" can be used to clear AX.

X	Y	X XOR Y
0	0	0
0	1	1
1	0	1
1	1	0

SECTION B.2: INSTRUCTION TIMING

In this section of the appendix we provide clock counts for all the instructions of Intel's 8086, 286, 386, and 486 microprocessors. The clock count is the number of clocks that it takes the instruction to execute. They are extracted from Intel's reference manuals on these microprocessors. The number of clocks for each instruction is given with the assumption that the instruction is already fetched into the CPU. The actual clock count can vary depending on the memory hardware design of the system. Note the following points when calculating the clock counts for a given CPU.

1. In calculating the total clock cycles for the 8086/88, one must add the extra clocks associated with the effective address (EA) provided in Table B-3.

2. In calculating the time required for the 8086, 286, and 386SX microprocessors, its 16-bit external data bus must be taken into consideration. In addition, whether the operand address is odd or even must be considered. To reduce the time required to fetch data from memory, these CPUs require that the data be aligned on even address boundaries. If addresses are not on even boundaries, an extra 4 clock-cycle penalty is added when fetching a 16-bit operand. Look at the following examples, assuming that DS = 2500H and BX = 3000H.

```
MOV    AX,[BX];total clocks = 10 + 5
```

Since the physical address is 25000H+3000H=28000H, an even address, and the data bus in the 8086 is 16 bits wide, the contents of memory locations 28000H and 28001H will be fetched into the CPU in one memory cycle. In all 80x86 microprocessors, the low byte goes to the low address and the high byte to the high address. In the example above, the contents of memory location 28000H will go to register AL and 28001H to AH. If BX = 3005H, the physical address would be 25000H + 3005H = 28005H, an odd location, and the clocks required would be as follows due to the extra 4 clock penalty for nonaligned data.

```
MOV    AX,[BX]          ;total clocks=10+4+5
```

In the instruction above the contents of memory location 28005 are moved to AL and 28006H to AH. In actuality, the way the 8086 accesses memory is that in the first memory cycle, the 16-bit data from 28004H and 28005H is accessed on the D0 - D15 data bus and then the 16-bit data of memory locations 28006H and 28007H is fetched in the second memory cycle using the 16-bit data bus. In other words, although memory locations 28004H, 28005H, 28006H, and 28007H were addressed by the 8086 in two consecutive memory cycles, only the contents of 28005H and 28006H are used; the contents of memory locations 28004H and 28007H are discarded. For this reason the data must be word (16-bit) aligned in the 16-bit data bus microprocessors. What happens if an odd address is accessed in the 8086? It still will take only one memory cycle consisting of 4 clocks. For example, in "MOV AH,[BX]" with BX = 3005H and DS = 25000H, the contents of memory locations 28004H and 28005H both are accessed with one memory cycle, but only the contents of address 28005H are fetched into register AH.

3. In the 8088 microprocessor, the time required to execute an instruction can vary from the 8086 since the data bus is only 8 bits in the 8088. A 16-bit operand would require two memory cycles (each consisting of 4 clock cycles) to move the operand in or out of the microprocessor: for example,

```
MOV    AL,[BX]          ;total clocks = 10 + 5

MOV    AX,[BX]          ;total clocks = 10 + 4 + 5
```

4. For conditional jumps and LOOP instructions, the first number is the number of clocks if the jump is successful (jump is taken) and the second number is for when the jump is not taken (noj = no jump). For example, the 8086 column for the JNZ instruction has "16,noj 4" for the clock count. The 16 is the clock cycle for the case when the jump is taken. If there is no jump, the clock is 4.

5. The clock number for the 80386SX is the same as the 80386, except for accessing 32-bit operands, for which an extra 2 clocks should be added since the data bus in the 80386SX is 16-bit and the memory cycle time of the 80386SX is 2 clocks.

6. An extra 2 clocks must be added for the 80286 and 80386SX if a 16-bit word operand is not aligned and also for the 386 if a 32-bit operand is not aligned at the 32-bit boundary. See the discussion above in point 2.

7. The number of clocks given for the 80486 microprocessor is for situations when the operand is in the cache memory of the 486 chip; otherwise, extra clocks should be added for the cache miss penalty. For the list of the cache miss penalties, refer to Intel's "i486 Microprocessor Programming Reference Manual."

8. PM (privilege mode) instruction timings are for situations when the CPU is switched to protected mode.

9. The "m" (often seen in 286 and 386 instructions) represents the number of components associated with the next instruction to be executed. The value of m varies because the size of the instruction located at the target address can vary. Generally, m can be averaged to 2.

10. The "n" represents the number of repetitions of a given instruction.

11. Due to the ever-advancing architectural design of the 286/386/486 microprocessors, the total clock count for a given program cannot be 100% correct. For this reason a 10% margin of error should be taken into consideration when calculating the total clock count of a given program.

12. With every new generation of 80x86, new instructions are added; therefore, there is no clock count for the prior generations. This indicated by "--".

Table B-3: Clock Cycles for Effective Address

Addressing Mode	Operand	CLK
Direct	label	6
Register indirect	[BX]	5
	[SI]	5
	[DI]	5
	[BP]	5
Based relative	[BX]+disp	9
	[BP]+disp	9
Indexed relative	[DI]+disp	9
	[SI]+disp	9
Based indexed	[BX][SI]	7
	[BX][DI]	7
	[BP][SI]	8
	[BP][DI]	8
Based indexed relative	[BX][SI]+disp	11
	[BX][DI]+disp	11
	[BP][SI]+disp	12
	[BP][DI]+disp	12

Note:
These times assume no segment override. If a segment override is used, 2 clock cycles must be added.

A summary of the clock cycles for various Intel microprocessors, by instruction, is given in Table B-4.

Table B-4: Clock Cycles for Various Intel Microprocessors by Instruction

Code	Description	8086	80286	80386	80486
AAA	ASCII adjust for addition	8	3	4	3
AAD	ASCII adjust for division	60	14	19	14
AAM	ASCII adjust for multiplication	83	16	17	15
AAS	ASCII adjust for subtraction	8	3	4	3
ADC	Add with carry				
	reg to reg	3	2	2	1
	mem to reg	9+EA	7	6	2
	reg to mem	16+EA	7	7	3
	immed to reg	4	3	2	1
	immed to mem	17+EA	7	7	3
	immed to acc	4	3	2	1
ADD	Addition				
	reg to reg	3	2	2	1
	mem to reg	9+EA	7	6	2
	reg to mem	16+EA	7	7	3
	immed to reg	4	3	2	1
	immed to mem	17+EA	7	7	3
	immed to acc	4	3	2	1
AND	Logical AND				
	reg to reg	3	2	2	1
	mem to reg	9+EA	7	6	2
	reg to mem	16+EA	7	7	3
	immed to reg	4	3	2	1
	immed to mem	17+EA	7	7	3
	immed to acc	4	3	2	1
ARPL	Adjust RPL (requested privilege level)				
	reg to reg	--	10	20	9
	reg to mem	--	11	21	9
BOUND	Check array bounds	--	13noj	10noj	7noj
BSF	Bit scan forward				
	reg to reg	--	--	10+3n	6/42
	mem to reg	--	--	10+3n	7/43
BSR	Bit scan reverse				
	reg to reg	--	--	10+3n	6/103
	mem to reg	--	--	10+3n	7/104
BSWAP	Byte swap	--	--	--	1
BT	Bit test				
	reg to reg	--	--	3	3
	reg to mem	--	--	12	8
	immed to reg	--	--	3	3
	immed to mem	--	--	6	3
BTC/ BTR/ BTS	Bit test complement/ Bit test reset/ Bit test set				
	reg to reg	--	--	6	6
	reg to mem	--	--	13	13
	immed to reg	--	--	6	6
	immed to mem	--	--	8	8

Table B-4: Clock Cycles for Various Intel Microprocessors by Instruction (continued)

Code	Description	8086	80286	80386	80486
CALL	Call a procedure				
	intrasegment direct	19	7+m	7+m	3
	intrasegment indirect through register	16	7+m	7+m	5
	instrasegment indirect through memory	21+EA	11+m	10+m	5
	intersegment direct	28	13+m	17+m	18
	486: to same level				20
	486: thru Gate to same level				35
	486: to inner level, no parameters				69
	486: to inner level, x parameter (d) words				77+4x
	486: to TSS				37+TS
	486: thru Task Gate				38+TS
	intersegment direct PM	--	26+m	34+m	
	intersegment indirect	37+EA	16+m	22+m	17
	486: to same level				20
	486: thru Gate to same level				35
	486: to inner level, no parameters				69
	486: to innter level, x parameter (d) words				77+4x
	486: to TSS				37+TS
	486: thru Task Gate				38+TS
	intersegment indirect PM	--	29+m	38+m	
CBW	Convert byte to word	2	2	3	3
CDQ	Convert double to quad	--	--	2	
CLC	Clear carry flag	2	2	2	2
CLD	Clear direction flag	2	2	2	2
CLI	Clear interrupt flag	2	3	3	5
CLTS	Clear task switched flag	--	2	5	7
CMC	Complement carry flag	2	2	2	2
CMP	Compare				
	reg to reg	3	2	2	1
	mem to reg	9+EA	6	6	2
	reg to mem	9+EA	7	5	2
	immed to reg	4	3	2	1
	immed to mem	10+EA	6	5	2
	immed to acc	4	3	2	1
CMPS/	Compare string/				
CMPSB/	Compare byte string/				
CMPSW	Compare word string				
	not repeated	22	8	10	8
	REPE/REPNE CMPS/CMPSB/CMPSW	9+22/rep	5+9/rep	5+9/rep	7+7c
CMPXCHG	Compare and exchange				
	reg with reg	--	--	--	6
	reg with mem	--	--	--	7/10
CWD	Convert word to doubleword	5	2	2	3
CWDE	Convert word to extended double	--	--	3	3
DAA	Decimal adjust for addition	4	3	4	2
DAS	Decimal adjust for subtraction	4	3	4	2

Table B-4: Clock Cycles for Various Intel Microprocessors by Instruction (continued)

Code	Description	8086	80286	80386	80486
DEC	Decrement by 1				
	16-bit reg	3	2	2	1
	8-bit reg	3	2	2	1
	memory	15+EA	7	6	3
DIV	Unsigned division				
	8-bit reg	80-90	14	14	16
	16-bit reg	144-162	22	22	24
	double	--	--	--	40
	8-bit mem	(86-96)+EA	17	17	16
	16-bit mem	(150-168)+EA	25	25	24
	double	--	--	--	40
ENTER	Make stack frame				
	W,0	--	11	10	14
	W,1	--	15	12	17
	dw,db	--	12+4(n-1)	15+4(n-1)	17+3n
ESC	Escape				
	reg	2	9-20	varies	
	mem	8+EA	9-20	varies	
HLT	Halt	2	2	5	4
IDIV	Integer division				
	8-bit reg	101-112	17	19	19
	16-bit reg	165-184	25	27	27
	32-bit reg	--	--	43	43
	8-bit mem	(107-118) +EA	20	22	20
	16-bit mem	(171-190) +EA	28	30	28
	32-bit reg	--	--	46	44
IMUL	Integer multiplication				
	8-bit reg	80-98	13	9-14	13-18
	16-bit reg	128-154	21	9-22	13-26
	32-bit reg	--	--	9-38	13-42
	8-bit mem	(86-104)+EA	16	12-17	13-18
	16-bit mem	(134-160)+EA	24	12-25	13-26
	32-bit reg	--	--	12-41	13-42
	immed to 16-bit reg	--	21	9-34	13-18
	immed to 32-bit reg	--	21	9-38	13-18
	reg to reg (byte)	--	--	9-38	13-18
	reg to reg (word)	--	--	9-38	13-26
	reg to reg (dword)	--	--	9-38	13-42
	mem to reg (byte)	--	--	12-25	13-18
	mem to reg (word)	--	--	12-25	13-26
	mem to reg (dword)	--	--	12-41	13-42
	reg with imm to reg (byte)	--	--	9-14	13-18
	reg with imm to reg (word)	--	--	9-22	13-26
	reg with imm to reg (dword)	--	--	9-38	13-42
	mem with imm to reg (byte)	--	--	12-17	13-18
	mem with imm to reg (word)	--	--	12-25	13-26
	mem with imm to reg (dword)	--	--	12-41	13-42

Table B-4: Clock Cycles for Various Intel Microprocessors by Instruction (continued)

Code	Description	8086	80286	80386	80486
IN	Input from I/O port				
	fixed port	10	5	12	14
	variable port through DX	8	5	13	14
INC	Increment by 1				
	16-bit reg	3	2	2	1
	8-bit reg	3	2	2	1
	mem	15+EA	7	6	3
INS/	Input from port to string				
INSB/	Input byte				
INSW/	Input word				
INSD	Input double	--	5	15	17
	PM	--	--	9,29	10-32
	REP INS/INSB/INSW	--	5+4/rep	13+6/rep	
	REP INS/INSB/INSW PM	--	--	(7,27)+6/rep	
INT	Interrupt				
	type=3	52	23+m	33	
	type=3 PM	--	(40,78)+m	59,99	
	type3	51	23+m	37	
	type3 PM	--	(40,78)+m	59,99	
INTO	Interrupt if overflow				
	interrupt taken	53	24+m	35	
	interrupt not taken	4	3	3	
	PM	--	(40,78)+m	59,99	
INVD	Invalidate data cache	--	--	--	4
INVLPG	Invalidate TLB entry	--	--	--	12/11
IRET	Return from interrupt	32	17+m	22	15
	PM	--	(31,55)+m	38,82	36
IRETD	Return from interrupt double	--	--	22	20
	PM	--	--	38,82	36
JA/	Jump if above/	16,noj 4	7+m,noj 3	7+m,noj 3	3,noj 1
JNBE	Jump if not below or equal				
JAE/	Jump if above or equal/	16,noj 4	7+m,noj 3	7+m,noj 3	3,noj 1
JNB	Jump if not below/				
JNA	Jump if not above				
JCXZ	Jump if CX is zero	18,noj 6	8+m,noj 4	9+m,noj 5	8,noj 5
JECXZ	Jump if ECX is zero	--	--	--	8,noj 5
JE/	Jump if equal/	16,noj 4	7+m,noj 3	7+m,noj 3	3,noj 1
JZ	Jump if zero				
JG/	Jump if greater	16,noj 4	7+m,noj 3	7+m,noj 3	3,noj 1
JNLE	Jump if not less, or equal				
JGE/	Jump if greater or equal/	16,noj 4	7+m,noj 3	7+m,noj 3	3,noj 1
JNL	Jump if not less				
JL/	Jump if less/	16,noj 4	7+m,noj 3	7+m,noj 3	3,noj 1
JNGE	Jump if not greater, or equal				
JLE/	Jump if less or equal/	16,noj 4	7+m,noj 3	7+m,noj 3	3,noj 1
JNG	Jump if not greater				

Table B-4: Clock Cycles for Various Intel Microprocessors by Instruction (continued)

Code	Description	8086	80286	80386	80486
JMP	Jump				
	intrasegment direct short	15	7+m	7+m	3
	intrasegment direct	15	7+m	7+m	3
	intersegment direct	15	11+m	12+m	17
	PM	--	23+m	27+m	18
	intrasegment indirect through memory	18+EA	11+m	10+m	5
	intrasegment indirect through register	11	7+m	7+m	5
	intersegment indirect	24+EA	15+m	12+m	8
	PM	--	26+m	27+m	18
	direct intersegment				17
	486: to same level				19
	486: thru call gate to same level				32
	486: thru TSS				42+TS
	486: thr Task Gate				43+TS
	indirect intersegment				13
	486: to same level				18
	486: thru call gate to same level				31
	486: thru TSS				41+TS
	486: thr Task Gate				42+TS
JNE/ JNZ	Jump if not equal/ Jump if not zero	16,noj 4	7+m,noj 3	7+m,noj 3	3,noj 1
JNO	Jump if not overflow	16,noj 4	7+m,noj 3	7+m,noj 3	3,noj 1
JNP/ JPO	Jump if not parity/ Jump if parity odd	16,noj 4	7+m,noj 3	7+m,noj 3	3,noj 1
JNS	Jump if not sign	16,noj 4	7+m,noj 3	7+m,noj 3	3,noj 1
JO	Jump if overflow	16,noj 4	7+m,noj 3	7+m,noj 3	3,noj 1
JP/ JPE	Jump if parity/ Jump if parity even	16,noj 4	7+m,noj 3	7+m,noj 3	3,noj 1
JS	Jump if sign	16,noj 4	7+m,noj 3	7+m,noj 3	3,noj 1
LAHF	Load AH from flags	4	2	2	3
LAR	Load access rights				
	reg to reg	--	14	15	11
	mem to reg	--	16	16	11
LDS/ LES	Load pointer using DS/ Load pointer using ES	16+EA	7	7	6
	PM	--	21	22	12
LFS/ LGS/ LSS	Load far pointer	--	--	7	6/12
	PM	--	--	22-25	
LEA	Load effective address	2+EA	3	2	2,noj 1
LEAVE	High level procedure exit	--	5	4	5
LGDT	Load global descriptor table	--	11	11	11
LIDT	Load interrupt desc. table	--	12	11	11
LLDT	Load local desc. table				
	reg	--	17	20	11
	mem	--	19	24	11
LMSW	Load machine status word				
	reg	--	3	10	13
	mem	--	6	13	13

Table B-4: Clock Cycles for Various Intel Microprocessors by Instruction (continued)

Code	Description	8086	80286	80386	80486
LOCK	Lock bus	2	0	0	1
LODS/	Load string/				
LODSB/	Load byte string/				
LODSW	Load word string				
	not repeated	12	5	5	5
	repeated	9+13/rep			7+4c
LOOP	Loop	17,noj 5	8+m,noj 4	11+m	7,noj 6
LOOPE/	Loop if equal/				
LOOPZ	Loop if zero	18,noj 6	8+m,noj 4	11+m	9,noj 6
LOOPNE/	Loop if not equal/				
LOOPNZ	Loop if not zero	19,noj 5	8+m,noj 4	11+m	9,noj 6
LSL	Load segment limit				
	reg to reg	--	14	20,25	10
	mem to reg	--	16	21,26	10
LTR	Load task register				
	reg	--	17	23	20
	mem	--	19	27	20
MOV	Move				
	acc to mem	10	3	2	1
	mem to acc	10	5	4	1
	reg to reg	2	2	2	1
	mem to reg	8+EA	5	4	1
	reg to mem	9+EA	3	2	1
	immed to reg	4	2	2	1
	immed to mem	10+EA	3	2	1
	reg to SS/DS/ES	2	2	2	3/9
	reg to SS/DS/ES PM	--	17	18	
	mem to SS/DS/ES	8+EA	5	5	3/9
	mem to SS/DS/ES PM	--	19	19	3
	segment reg to reg	2	2	2	3
	segment reg to mem	9+EA	3	2	3
	control reg to reg	--	--	6	4
	reg to control reg 0	--	--	10	16
	reg to control reg 2	--	--	4	4
	reg to control reg 3	--	--	5	4
	debug reg 0-3 to reg	--	--	22	10
	debug reg 6-7 to reg	--	--	14	10
	reg to debug reg 0-3	--	--	22	11
	reg to debug reg 6-7	--	--	16	11
	test reg to reg	--	--	12	3,4
	reg to test reg	--	--	12	6,4
MOVS/	Move string/				
MOVSB/	Move byte string/				
MOVSW	Move word string				
	not repeated	18	5	7	7
	REP MOVS/MOVSB/MOVSW	9+17/rep	5+4/rep	8+4/rep	12+3/rep
MOVSX	Move with sign-extend				
	reg to reg	--	--	3	3
	mem to reg	--	--	6	3
MOVZX	Move with zero-extend				
	reg to reg	--	--	3	3
	mem to reg	--	--	6	3

Table B-4: Clock Cycles for Various Intel Microprocessors by Instruction (continued)

Code	Description	8086	80286	80386	80486
MUL	Unsigned multiplication				
	8-bit reg	70-77	13	9-14	13/18
	16-bit reg	118-133	21	9-22	13/26
	double	--	--	9-38	13/42
	8-bit mem	(76-83)+EA	16	12-17	13/18
	16-bit mem	(124-139)+EA	24	12-25	13/26
	double	--	--	12-41	13/42
NEG	Negate				
	reg	3	2	2	1
	mem	16+EA	7	6	3
NOP	No operation	3	3	3	3
NOT	Logical NOT				
	reg	3	2	2	1
	mem	16+EA	7	6	3
OR	Logical OR				
	reg to reg	3	2	2	1
	mem to reg	9+EA	7	6	2
	reg to mem	16+EA	7	7	3
	immed to acc	4	3	2	1
	immed to reg	4	3	2	1
	immed to mem	17+EA	7	7	3
OUT	Output to I/O port				
	fixed port	10	3	10	16
	fixed port PM	--	--	4,24	11,31
	variable port	8	3	11	16
	variable port PM	--	--	5,25	10,30
OUTS/	Output string to port/				
OUTSB/	Output byte				
OUTSW/	Output word				
OUTSD	Output double	--	5	14	17
	PM	--	--	8,28	10,32
	REP OUTS/OUTSB/OUTSW	--	5+4/rep	12+5/rep	
	REP OUTS/OUTSB/OUTSW PM	--	--	(6,26)+5/rep	
POP	Pop word off stack				
	reg	8	5	4	4
	segment reg	8	5	7	3/9
	segment reg PM	--	20	21	9
	memory	17+EA	5	5	6
POPA/	Pop all			9	
POPAD	Pop all double	--	19	24	9
POPF	Pop flags off stack	8	5	5	9
POPFD	Pop flags off stack double	--	--	5	
PUSH	Push word onto stack				
	reg	11	3	2	4
	segment reg: ES/SS/CS	10	3	2	3
	segment reg: FS/GS	--	--	2	3
	memory	16+EA	5	5	4
	immed	--	3	2	1
PUSHA	Push All	--	17	18	11
PUSHF/	Push flags onto stack				
PUSHD	Push double flag onto stack	10	3	4	4

Table B-4: Clock Cycles for Various Intel Microprocessors by Instruction (continued)

Code	Description	8086	80286	80386	80486
RCL/	Rotate left through carry/				
RCR	Rotate right through carry/				
	reg with single-shift	2	2	9	3
	reg with variable-shift	8+4/bit	5+n	9	8/30
	mem with single-shift	15+EA	7	10	4
	mem with variable-shift	20+EA+4/bit	8+n	10	9/31
	immed to reg	--	5+n	9	8/30
	immed to mem	--	8+n	10	9/31
RET/	Return from procedure/				
RETF/	Return far/				
RETN	Return near				
	intrasegment	16	11+m	10+m	5
	intrasegment with constant	20	11+m	10+m	5
	intersegment	26	15+m	18+m	18
	intersegment PM	--	25+m,55	32+m,62	13
	intersegment with constant	25	15+m	18+m	33
	intersegment w/constant PM	--	25+m,55	32+m,68	17
	486: imm. to SP	--	--	--	14
	486: to same level	--	--	--	17
	486: to outer level	--	--	--	33
ROL/	Rotate left				
ROR	Rotate right				
	reg with single-shift	2	2	3	3
	reg with variable-shift	8+4/bit	5+n	3	3
	mem with single-shift	15+EA	7	7	4
	mem with variable-shift	20+EA+4/bit	8+n	7	4
	immed to reg	--	5+n	3	2
	immed to mem	--	8+n	7	4
SAHF	Store AH into flags	4	2	3	2
SAL/	Shift arithmetic left/				
SAR/	Shift arighmetic right/				
SHL/	Shift logical left/				
SHR	Shift logical right				
	reg with single-shift	2	2	3	3
	reg with variable-shift	8+4/bit	5+n	3	3
	mem with single-shift	15+EA	7	7	4
	mem with variable-shift	20+EA +4/bit	8+n	7	4
	immed to reg	--	5+n	3	2
	immed to mem	--	8+n	7	4
SBB	Subtract with borrow				
	reg from reg	3	2	2	1
	mem from reg	9+EA	7	7	2
	reg from mem	16+EA	7	6	3
	immed from acc	4	3	2	1
	immed from reg	4	3	2	1
	immed from mem	17+EA	7	7	3
SCAS/	Scan string/				
SCASB/	Scan byte string/				

Table B-4: Clock Cycles for Various Intel Microprocessors by Instruction (continued)

Code	Description	8086	80286	80386	80486
SCASW	Scan word string				
	not repeated	15	7	7	6
	REPE/REPNE SCAS/SCASB/SCASW	9+15/rep	5+8/rep	5+8/rep	7+5/rep
SET	Set conditionally				
	reg	--	--	4	4 or 3
	mem	--	--	5	3 or 4
SGDT	Store global descript. table	--	11	9	10
SIDT	Store interrupt desc. table	--	12	9	10
SLDT	Store local desc. table				
	reg	--	2	2	2
	mem	--	3	2	3
SHLD/	Shift left double precision/				
SHRD	Shift right double				
	reg to reg	--	--	3	2
	mem to mem	--	--	7	3
	reg by CL	--	--	3	3
	mem by CL	--	--	7	4
SMSW	Store machine status word				
	reg	--	2	10	2
	mem	--	3	3	3
	mem PM	--	--	2	
STC	Set carry flag	2	2	2	2
STD	Set direction flag	2	2	2	2
STI	Set interrupt flag	2	2	3	5
STOS/	Store string/				
STOSB/	Store byte string/				
STOSW	Store word string				
	not repeated	11	3	4	5
	REP STOS/STOSB/STOSW	9+10/rep	4+3/rep	5+5/rep	7+4/rep
STR	Store task register				
	reg	--	2	2	2
	mem	--	3	2	3
SUB	Subtraction				
	reg from reg	3	2	2	1
	mem from reg	9+EA	7	7	2
	reg from mem	16+EA	7	6	3
	immed from acc	4	3	2	1
	immed from reg	4	3	2	1
	immed from mem	17+EA	7	7	3
TEST	Test				
	reg with reg	3	2	2	1
	mem with reg	9+EA	6	5	2
	immed with acc	4	3	2	1
	immed with reg	5	3	2	1
	immed with mem	11+EA	6	5	2
VERR	Verify read				
	reg	--	14	10	11
	mem	--	16	11	11
VERW	Verify write				
	reg	--	14	15	11
	mem	--	16	16	11

Table B-4: Clock Cycles for Various Intel Microprocessors by Instruction (continued)

Code	Description	8086	80286	80386	80486
WAIT	Wait while TEST pin				
	not asserted	4	3	6	1-3
WBINVD	Write-back invalid data cache	--	--	--	5
XADD	Exchange and add				
	reg with reg	--	--	--	3
	reg with mem	--	--	--	4
XCHG	Exchange				
	reg with acc	3	3	3	3
	reg wtih mem	17+EA	5	5	5
	reg with reg	4	3	3	3
XLAT/ XLATB	Translate	11	5	5	4
XOR	Logical exclusive OR				
	reg with reg	3	2	2	1
	mem with reg	9+EA	7	7	2
	reg wtih mem	16+EA	7	6	3
	immed with acc	4	3	2	1
	immed with reg	4	3	2	1
	immed with mem	17+EA	7	7	3